

1. (15 pts) (True or False questions) Please mark by $\bigcirc$ (True) or $\times$ (False) each of the following statements. You don't need to prove or argue it.

(是非題) 請用 $\bigcirc$ (對) 或 $\times$ (錯) 標記以下每個敘述，不需要證明。

- (1) A stack is a container of objects that are inserted and removed according to the first-in first-out (FIFO) principle.
堆疊是依照先進先出 (FIFO) 原則插入和刪除物件的容器。
- (2) The `dequeue` operation at the front of a general n -element array-based queue Q takes $O(n)$ time if we use $Q[0]$ to be the front always.
如果我們固定使用 $Q[0]$ 作為前頭游標，那麼在一般以陣列實現的 n 個元素佇列 Q 的前頭進行 `dequeue` 操作需要 $O(n)$ 時間。
- (3) Consider a circular array-based queue and let two variable f and r denote the indexes of the front and element immediately after the rear, respectively. The condition $f = r$ says that this queue is empty.
考慮一個基於循環陣列的佇列，若 f 和 r 分別表示前頭元素和緊接在尾端元素的索引，條件 $f = r$ 表示該佇列為空。
- (4) When using Polish notation for mathematical expression, the parentheses are necessary for overriding the precedence of operators.
當使用 Polish 表示法進行數學表示式時，需要使用括號來界定運算子的優先權。
- (5) Removing at the tail of a singly linked list takes a linear time in worse case.
在最壞的情況下，刪除單向鏈結串列的尾端節點需要線性時間。

1. False (FILO)

2. True (左移動每個元素)

3. False (有可能是滿的)

4. False (Prefix 不用, 只有 infix 需要)

5. True (要從 head 跑到 tail)

2. (15 pts) (True or False questions) Please mark by ○(True) or ×(False) each of the following statements. You don't need to prove or argue it.
(是非題) 請用 ○(對) 或 ×(錯) 標記以下每個敘述，不需要證明。
- (1) Let T be a B-tree of order 11, the possible degrees of internal nodes (except the root) are 6, 7, 8, 9, 10 and 11.
設 T 為一棵11階B樹，內部節點(根除外)可能的分支量(degree)為6、7、8、9、10和11。
 - (2) When deleting a node from an AVL tree, the height-balance property may need more than one restructuring to be restored.
當從AVL樹中刪除節點時，高度平衡屬性可能需要多次重構(restructuring)才能恢復。
 - (3) The `insert` operation takes $O(1)$ time on a dictionary implemented by means of an unsorted list.
在透過未排序串列實現的字典上，`insert`操作需要 $O(1)$ 時間。
 - (4) The preorder and postorder traversal can UNIQUELY define a binary tree.
前序和後序走訪可以定義出唯一二元樹。
 - (5) Suppose one uses an sorted list to implement a priority queue P . Then, the sort using P is known as Selection sort.
假設 P 為使用排序串列來實現優先佇列，則使用 P 進行的排序方法為選擇排序。

1. True Children 數量是 $\text{ceil}(m/2) \sim m$

2. True Insert: $O(1)$ 次旋轉
Remove: $O(\log n)$ 次旋轉

3. True 直接append

4. False 需要 [In-order + Pre-order] 或 [In-order + Post-order]

5. True

3. (15 pts) Please answer the following problems simply.

請簡單回答以下問題。

(1) Please write the following infix expression in postfix form:

請以後序表示式寫出下面的中序表示式：

$$(a - b * c / d * e + f - g) * h$$

(2) Please write the following prefix expression in infix form:

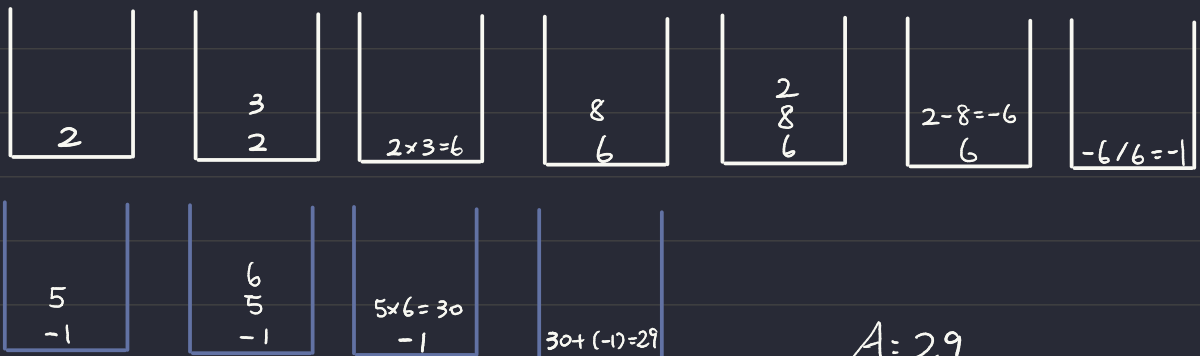
請以中序表示式寫出下面的前序表示式：

$$/*4a - /*/*bc + abdc$$

(3) Draw the sequence of stack configurations for the evaluation of the following postfix expression. Assume that $a = 2$, $b = 3$, $c = 8$, $d = 5$, and $e = 6$.

假設 $a = 2$ 、 $b = 3$ 、 $c = 8$ 、 $d = 5$ 和 $e = 6$ ，我們使用堆疊幫助計算以下後序表示式的值，請以計算過程依序畫出堆疊的內容。

$$ab * ca - / de * +$$



$$A: 29$$

4. (15 pts) Consider the following keys 18, 24, 7, 10, 21, and 4 and we would like to build a binary search tree T using a sequence of insertions with the given order. Please answer the following questions concisely and shortly.

考慮以下鍵值18, 24, 7, 10, 21, 及4, 我們希望使用給定的順序以連續的插入操作來建立二元搜尋樹 T 。請簡明扼要地回答下列問題：

- (1) Suppose the height of a leaf node is 0. What is the height of the generated binary search tree T ?

假設葉節點的高度為0, 則產生的二元搜尋樹 T 的高度是多少?

- (2) T is a proper binary tree. (○(True) or ×(False))

T 是一顆純二元樹。○(對)或×(錯)?

- (3) Suppose we insert key 15 to T and derive a new binary search tree T' . Which node is the parent of the node with key 15 in T' ?

假設我們將鍵值15插入 T 並衍生出一個新的二元搜尋樹 T' , 在 T' 中, 哪一個節點是鍵值15的節點的父節點?

- (4) From T' , after removing key 7, the resulting binary search tree T'' is a complete binary tree. (○(True) or ×(False))

從 T' 開始, 刪除鍵值7後, 得到的二元搜尋樹 T'' 是一個完全二元樹。○(對)或×(錯)?

- (5) Which traversal can be used to list all the keys in a sorted order on a binary search tree?

哪一種走訪可用於在二元搜尋樹上按排序順序列出所有鍵值?

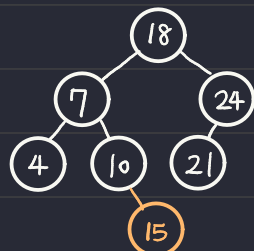


A: 2

(2)

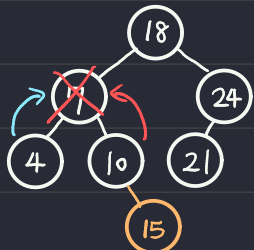
A: False

(3)



A: 10

(4)



找左側最大 or 右側最小遞補

successor / predecessor

A: False

(5)

A: In order

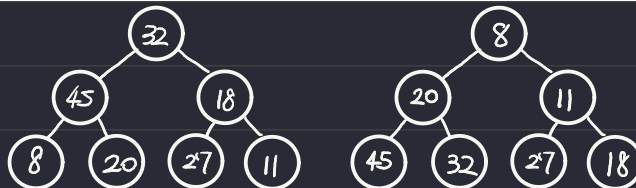
5. (15 pts) Suppose that we have the following keys 32, 45, 18, 8, 20, 27 and 11. We place these key in the same order one by one to an array A starting at position(index) 1. Then, array A can be as a complete binary tree T conceptually. With complete binary tree T , we can use **Heapify** operation to have the associated min_heap (minumum heap) H .

Please answer the following questions concisely and shortly.

假設我們有以下鍵值32、45、18、8、20、27和11。我們將這些鍵值依照給出的順序一一從位置(索引) 1 開始放入陣列 A 中，在概念上 A 可以作為一顆完全二元樹 T 。有了完全二元樹 T ，我們可以使用**Heapify**操作來獲得最小堆積(min_heap) H 。請簡明扼要地回答下列問題：

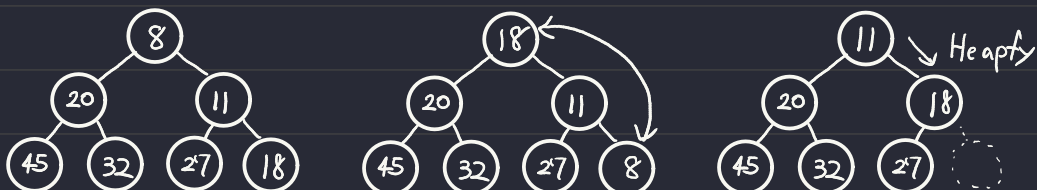
- (1) Which node is the last node in H ?
 H 中最後節點是哪個節點
- (2) After we perform remove_Min operation on H , which node is the left child of the node with key 11?
我們對 H 執行remove_Min操作後，哪個節點是鍵為值11的節點的左子節點？
- (3) Consider that we build the min_heap H' using a sequence of insertion operations with the same order given above. Please list all the keys in the underlying array of H' from index 1 to index 7. Suppose that we implement the min_heap with an array.
我們使用與給出順序相同的次序進行一系列插入操作來建立另一最小堆積(min_heap) H' 。若我們以陣列來實現最小堆積，請列出相對應於 H' 的底層陣列中從索引1到索引7的所有鍵值。
- (4) A heap can be constructed in linear time if we have all the keys. ($\bigcirc$ (True) or $\times$ (False))
如果我們擁有所有的鍵值，則可以在線性時間內建立出一個堆積。 $\bigcirc$ (對)或 $\times$ (錯)？
- (5) The way we use a heap to sort a group of keys is known as heapsort. What is the worst case running time of heapsort with n elements.
我們使用堆積對一組鍵值進行排序的方式稱為堆積排序。具有 n 個元素的堆積排序其最壞情況運行的時間是多少？

(1)



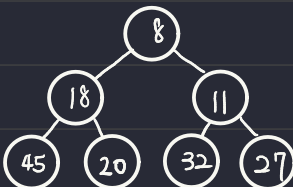
Ans: 18

(2)



Ans: 20

(3)



ANS: 8, 18, 11, 32, 20, 45, 27

(4)

Use static array = $O(n)$

ANS: True

(5)

ANS: $O(n \log n)$

6. (10 pts) A *d-ary heap* is like a binary heap except that non-leaf nodes have d children instead of 2 children. Please answer the following questions.

d-ary heap 類似二元堆積，只不過非葉節點有 d 個子節點而非2個子節點。請回答以下問題：

(1) How to represent a d -ary heap in an array?

如何在陣列中表示 d 元堆積？

(2) What is the height of a d -ary heap of n elements in terms of n and d ?

請以 n 和 d 表示的 n 個元素的 d 元堆積的高度是多少？

(1) 父節點： $\lfloor \frac{i-1}{d} \rfloor$

子節點： $i \cdot d + 1 \sim i \cdot d + d$

(2) $\log_d n$

7. (5 pts) Recall the definition of a binary tree. We now say that a binary tree T is *balanced* if every node v in T , the heights of the left and right subtrees of v differ by at most 1. Define N_h to be the minimum number of nodes in a balanced binary tree of height h . Suppose $f_0 = 1, f_1 = 1, f_2 = 2, \dots, f_k = f_{k-1} + f_{k-2}, \dots$ denotes the Fibonacci sequence. Please show that

請回想一下二元樹的定義。當看到一個二元樹 T 當中，每一個節點 v 的左子樹和右子樹的高度相差最多為1時，我們稱 T 為平衡的。定義 N_h 為一高度為 h 的平衡二元樹的最小節點數。假定 $f_0 = 1, f_1 = 1, f_2 = 2, \dots, f_k = f_{k-1} + f_{k-2}, \dots$ 表示了費式數列。請證明

$$N_h = f_{h+2} - 1, \text{ for } h \geq 0.$$

- $N_h = N_{h-1} + N_{h-2} + 1$

(1) Assumption

$$N_h = f_{h+2} - 1$$

(2) Base Case

$$h=0 \quad N_0 = f_2 - 1 = 1 \quad \checkmark$$

$$h=1 \quad N_1 = f_3 - 1 = 2 \quad \checkmark$$

(3) Inductive Step.

$$N_h = N_{h-1} + N_{h-2} + 1$$

$$= (f_{h+1} - 1) + (f_h - 1) + 1$$

$$= f_{h+1} + f_h + 1$$

$$= f_{h+2} + 1 \quad \#$$

8. (10 pts) Consider the red-black tree in Figure 1.

考量如圖 1 中的紅黑樹。

(1) Please construct a corresponding (2,4)-tree.

請建構出一個對應的(2,4)樹。

(2) Use the (2,4)-tree derived in (1) and perform the following sequence of insertions:

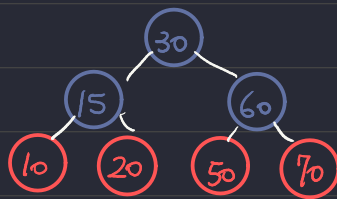
`insert(63)`, `insert(28)` and `insert(18)`.

使用在子題(1)所得到的(2,4)樹，依序進行下列加入的操作，請標示執行動作。

`insert(63)`、`insert(28)`、`insert(18)`

You need to mark the actions taken at each step. No partial credits will be given to a wrong input for each subproblem.

你必須在每一步標示出其動作，而每一子題的input皆須正確，否則不予計分。



(1)



(2)

